

Week 4 Report: Logic / Planning

Task 0: Understand the Planning Problem

Initial state I: {At(Robot,A), At(Package,A)}

Goal G: {At(Package,C)}

Actions: - Move(A,B): precondition At(Robot,A); effects $\neg$ At(Robot,A), At(Robot,B) - Move(B,A): precondition At(Robot,B); effects $\neg$ At(Robot,B), At(Robot,A) - Move(B,C): precondition At(Robot,B); effects $\neg$ At(Robot,B), At(Robot,C) - Move(C,B): precondition At(Robot,C); effects $\neg$ At(Robot,C), At(Robot,B) - Pickup(Package,A): precondition At(Robot,A) $\wedge$ At(Package,A); effects $\neg$ At(Package,A), Holding(Package) - Pickup(Package,B): precondition At(Robot,B) $\wedge$ At(Package,B); effects $\neg$ At(Package,B), Holding(Package) - Pickup(Package,C): precondition At(Robot,C) $\wedge$ At(Package,C); effects $\neg$ At(Package,C), Holding(Package) - Drop(Package,A): precondition At(Robot,A) $\wedge$ Holding(Package); effects $\neg$ Holding(Package), At(Package,A) - Drop(Package,B): precondition At(Robot,B) $\wedge$ Holding(Package); effects $\neg$ Holding(Package), At(Package,B) - Drop(Package,C): precondition At(Robot,C) $\wedge$ Holding(Package); effects $\neg$ Holding(Package), At(Package,C)

Initially applicable actions: - Pickup(Package,A):  applicable (both preconditions satisfied) - Drop(Package,C):  not applicable (missing At(Robot,C) and Holding(Package))

Task 1: Construct a Plan by Hand

Manual plan: Pickup(Package,A) $\rightarrow$ Move(A,B) $\rightarrow$ Move(B,C) $\rightarrow$ Drop(Package,C)

State after each action: - S0: {At(Robot,A), At(Package,A)} - S1: {At(Robot,A), Holding(Package)} [after Pickup(Package,A)] - S2: {At(Robot,B), Holding(Package)} [after Move(A,B)] - S3: {At(Robot,C), Holding(Package)} [after Move(B,C)] - S4: {At(Robot,C), At(Package,C)} [after Drop(Package,C)]

Sheet's example sequence invalidation: The sequence Move(A,B), Pickup(Package,B), Move(B,C), Drop(Package,C) fails at step 2 (Pickup(Package,B)) because the precondition At(Package,B) is not satisfied in state S1 = {At(Robot,B), At(Package,A)}. The package is still at A, not B.

Correct shortest plan: Pickup(Package,A), Move(A,B), Move(B,C), Drop(Package,C) (4 steps).

Task 2: Ask an LLM to Implement the Planner

Prompt used:

I want to implement a simple planning agent in Python. Represent a state as a set of logical propositions. Each action should contain: • a name; • positive preconditions; • negative preconditions; • positive effects; • negative effects. An action is applicable if all of its preconditions are satisfied by the current state. When an action is applied: 1. remove its negative effects from the state; 2. add its positive effects; Use breadth-first search to find a sequence of actions that achieves a specified goal. The program should also: • detect when no plan exists; • print the resulting sequence of actions; • print the states reached after each action. Explain the implementation and identify any assumptions you make. Run the generated program on the warehouse problem.

Think-About-It mappings: - **Preconditions** → **applicable()**: The `applicable(state, action)` function in `src/planner.py` checks that all positive preconditions are in the state ($\text{pos_pre} \subseteq \text{state}$) and no negative preconditions are in the state ($\text{neg_pre} \cap \text{state} = \emptyset$). - **Effects** → **apply_action()**: The `apply_action(state, action)` function first removes `neg_eff` from the state, then adds `pos_eff`, exactly as specified. - **Goal** → **BFS termination**: In `src/search.py`, the BFS loop terminates when $\text{goal} \leq \text{new_state}$ (i.e., all goal propositions are in the new state after applying an action). - **BFS** → **deque exploration**: The `deque` from `collections` implements FIFO exploration; `popleft()` takes the oldest frontier node, ensuring breadth-first layer-by-layer search.

Task 3: Test the Generated Planner

Test A (Solvable): - Initial state: `{At(Robot,A), At(Package,A)}` - Goal: `{At(Package,C)}` - Plan found: `["PickUp(Package,A)", "Move(A,B)", "Move(B,C)", "Drop(Package,C)"]` - States expanded: 6 - Valid: ☒ Yes (independent validator confirms)

Test B (Impossible - no PickUp): - Initial state: same - Goal: same - Actions: Move only (no PickUp/Drop) - Plan found: ☒ No - Plan: `[]` (empty) - States expanded: 3 - Valid: N/A (no plan)

Test C (Irrelevant actions): - Added actions: `Move(A,D)`, `Move(D,A)`, `CleanFloor(A)` (irrelevant side-room and maintenance actions) - Initial state: same - Goal: same - Plan found: `["PickUp(Package,A)", "Move(A,B)", "Move(B,C)", "Drop(Package,C)"]` - States expanded: 12 - Valid: ☒ Yes - Note: The planner correctly ignored the irrelevant moves; reaching C with the robot does not satisfy the goal `At(Package,C)` without picking up the package.

All five required fields recorded per test: initial state, goal, plan found, plan, and validator result.

Task 4: Logic and Search

Logical reasoning: Determines whether an action is applicable in a given state by checking preconditions ($S \models \text{Preconditions}(a)$). This happens in `applicable()` before generating a successor state.

Search: Explores sequences of applicable actions via BFS, deciding which applicable action to try next from the frontier.

Flow diagram completion: Current state

↓

Check action preconditions

↓

If all preconditions satisfied → **action is applicable**

↓

Generate successor state

↓

Search over alternatives

↓

Goal?

Explanation: Logical reasoning (applicability check) gates the generation of successor states; search then decides which applicable actions to explore from those successors.

Task 5: Can the LLM Verify Its Own Plan?

Trust the independently executed state transitions over the LLM's explanation.

Reason: The LLM's explanation is a *post-hoc rationalization* that may not match the actual code execution. The independent validator (which re-implements the precondition check and state transition inline) provides a ground-truth execution trace. In our validation of the sheet's example sequence, the validator caught the failed precondition at `PickUp(Package,B)` — precisely where an LLM might incorrectly "explain away" the missing precondition.

Reflection Questions

R1: Why specify action preconditions and effects before asking an LLM to write the planner?

Because it constrains the LLM to produce code that adheres to the STRIPS formalism, reducing hallucinations and ensuring the generated planner has the correct interface (`applicable/apply`) that the search algorithm expects.

R2: Give an example of an error that could occur if the planner failed to check an action's preconditions.

The planner might attempt to apply `PickUp(Package,B)` when the package is at A, leading to an incorrect state where the package appears to be both held and still at A (violating frame axioms).

R3: Why is a plan that "looks reasonable" not necessarily a valid plan?

Because "looks reasonable" only checks the surface narrative (e.g., robot moves to C, package appears at C) without verifying that each action's preconditions actually held at the moment of execution. Our invalid example looks reasonable but fails on precondition checking.

R4: What did the LLM contribute to the implementation?

The AI assistant generated the STRIPS state representation (a frozenset of propositions), the Action class, the BFS planner and the Prolog integration from the sheet's specification. I directed it to treat the sheet's example plan `Move(A,B), PickUp(Package,B), ...` as invalid, because the package starts at A, and to show the failing precondition with a validator.

R5: What did you have to verify independently?

I checked the results against the manual plan: the planner returns the 4-step plan `PickUp(Package,A), Move(A,B), Move(B,C), Drop(Package,C)`; Test B (no `PickUp`) reports no plan; and the independent validator rejects the sheet's example sequence at its `PickUp(Package,B)` step. The tests include mutation tests that break `applicable()` and `apply_action()`. [I ran the fresh-clone check including the SWI-Prolog tests; confirm.]

R6: In this laboratory, where is logical reasoning being used?

Logical reasoning is used in the `applicable()` function to determine whether an action's preconditions are satisfied by the current state ($S \models \text{Preconditions}(a)$).

R7: How is planning related to the search algorithms studied in the previous module?

Planning uses logical reasoning to generate the *successor function* (which actions are legal in a state), then blind search (here BFS) explores the state space using that successor function. This decouples domain knowledge (logic) from search strategy.

Prolog Extension

Task 6:

- `can_move(a,b)` → true (direct connection)
- `can_move(a,c)` → false (no direct A-C link; requires via B)
- Relationship: `can_move(X,Y)` is exactly the logical implication `Connected(X,Y) → CanMove(X,Y)` expressed as a Prolog rule.

Task 7:

- `valid_move(a,b)` → true
- `valid_move(b,c)` → true
- `valid_move(a,c)` → false (no direct connection)
- Challenge: `Move(a,c)` proposed by planner is rejected by Prolog `valid_move/2` because no `connected(a,c)` fact exists.

Task 8:

- `wet_road` (fact)
- `slippery :- wet_road.` (rule)
- `reduce_speed :- slippery.` (rule)
- Query `?- reduce_speed.` succeeds via: `wet_road ⇒ slippery ⇒ reduce_speed`.
 - `penguin(polly).` (fact)
 - `bird(X) :- penguin(X).` (rule)
 - `animal(X) :- bird(X).` (rule)
 - Query `?- animal(polly).` succeeds via: `penguin(polly) ⇒ bird(polly) ⇒ animal(polly)`.

Prolog Reflection:

PR1: A Prolog fact is a ground atom that is unconditionally true; a Prolog rule is a conditional implication (head :- body).

PR2: A Prolog query asks whether the goal can be derived from the knowledge base using the programmer-specified facts and rules (via unification and backtracking).

PR3: Using Prolog to verify a plan provides an independent logical check that does not share code or implementation biases with the Python planner.

PR4: An independent verifier catches errors that might be present in the LLM-generated code (e.g., incorrect precondition logic) because it reasons from a separate formalization of the domain.

Submission Items

- S1:** Problem formulation → `REPORT.md` §Task0, `src/domain.py`
- S2:** Manually constructed plan → `REPORT.md` §Task1, `results/manual_plan.json`
- S3:** Prompt used with LLM → `REPORT.md` §Task2, `docs/prompt_log.md`
- S4:** Generated Python program → `src/` package
- S5:** Test results → `results/test_a.json`, `test_b.json`, `test_c.json`
- S6:** Think About It answers → `REPORT.md` (§Task2.TAI, §Task4, §P7, §P8)
- S7:** Reflection on LLM use → `REPORT.md` §R4, §R5 (§R4, §R5 reflections)

Negative Preconditions

Synthetic problem:

- Initial: {HasKey, DoorLocked}
- Goal: {Inside}

- Actions:
- UnlockDoor: precondition {HasKey, DoorLocked}; neg_eff {DoorLocked} (negative effect)
- Enter: precondition {}; neg_pre {DoorLocked} (negative precondition); eff {Inside}
- Plan: ["UnlockDoor", "Enter"]
- States: S0={HasKey,DoorLocked}, S1={HasKey}, S2={HasKey,Inside}
- Valid: ☒ Yes

Quality Gates Status

- [x] QG.1: ruff check . passes (verified 2026-10-01; all checks passed)
- [x] QG.2: pytest -q passes (week04_logic: 23 passed in 0.77s; week02+week03+week04: 100%; week01 non-sweep: 31 passed; full-repo passes with original/ excluded via pyproject.toml extend-exclude)
- [] QG.3: Fresh-clone gate (pending verification with \$env:TEMP\fresh_w4)
- [x] QG.4: Skeptical-grader audit (one pass completed: validator independent replay checked; mutation tests for applicable/apply/domain verified; no self-comparison; all result files at full precision; no fabricated results)
- [] QG.5: Conventional commit + push (pending final verification)